

FLAG LAB

Dissertation

A Formal Verification Framework for CRDTs in Athena

Leonardo Ángel

December 6, 2025

This thesis is submitted in partial fulfillment of the requirements for a degree of Master in Systems and Computing Engineering (MISIS).

Thesis Committee:

Prof. Nicolás Cardozo (Promotor) Universidad de los Andes, Colombia

A Formal Verification Framework for CRDTs in Athena

© 2025 Leonardo Ángel

Systems and Computing Engineering Department

FLAG lab

Faculty of Engineering

Universidad de los Andes

Bogotá, Colombia

This manuscript has been set with the help of TEXSHOP and PDFLATEX (with BIBTEX support). Bugs have been tracked down in the text and squashed thanks to *Bugs in Writing* by Dupré [2] and *Elements of Style* by Strunk and White [14].

DEDICATION

Abstract

Conflict-Free Replicated Data Types (CRDTs) enable distributed systems to achieve eventual consistency without coordination, making them necessary for applications such as collaborative editors, distributed databases and decentralized systems. Designing CRDTs is complex: subtle choices in merge semantics can lead to unintended divergence, and proving correctness properties such as convergence, commutativity, and idempotence requires rigorous reasoning. While existing work demonstrates that individual CRDTs can be verified using theorem provers, each verification effort typically starts from scratch, requiring substantial proof overhead for common properties.

Our proposal offers a structured approach for CRDT verification by developing a modular framework in the Athena proof assistant. The framework facilitates the specification of both the syntax and operational semantics of CRDTs and guides the proof of key properties such as convergence, commutativity, and idempotence. By identifying and encapsulating common patterns in CRDT design and verification. It is suited to developers who are exploring or prototyping new CRDT variants and need a precise and checkable way to reason about their correctness. While the framework does not cover all possible CRDT designs. It takes a modest step toward making formal verification more approachable in this domain.

Contents

Abstract . . . iv
1 Introduction . . . 1
2 Related work . . . 3
2.1 Conflict-Free Replicated Data Types and eventual consistency . . . 3
2.1.1 Classification of CRDTs: State-based and Operation-based Approaches . . . 4
2.2 Formal specification and verification frameworks . . . 5
2.3 Byzantine fault tolerance and CRDTs . . . 6
2.4 Verification in actor and concurrency frameworks . . . 7
2.4.1 Program Synthesis and Automated CRDT Generation . . . 7
2.4.2 Comparative Analysis: Defining Grammar, Rule Inference, and Formal Verification . . . 8
2.4.3 Discussion . . . 8
2.4.4 Conclusion . . . 9
3 Athena in a Nutshell . . . 11
3.1 Why Athena for CRDT Verification? . . . 11
3.2 Basic Constructs . . . 11
3.2.1 Modules and Namespaces . . . 11
3.2.2 Sorts and Declarations . . . 12
3.2.3 Assertions: Stating Axioms . . . 13
3.2.4 Proofs: The `conclude` Construct . . . 13
3.3 Proof Methods . . . 14
3.3.1 Equational Reasoning with `!chain` . . . 14
3.3.2 Implication Chain with `!chain->` . . . 14
3.3.3 Universal Instantiation with `!uspec` . . . 15
3.3.4 Case Analysis with `!cases` . . . 16
3.4 Theories and Adapters . . . 16
3.4.1 Defining a Theory . . . 16
3.4.2 Instantiating via Adapters . . . 17
3.5 Reading Athena Code . . . 17
3.6 Tools and Workflow . . . 18
4 Theoretical Foundations . . . 19
4.1 Semilattices . . . 19
4.2 State-based CRDT model . . . 22
4.3 Concrete CRDT theories in Athena . . . 24
5 Framework Architecture . . . 29
5.1 Overview of the Layered Architecture . . . 29
5.2 Theory Layer: `SemilatticeTheory` and Abstract CRDT . . . 30

5.3 Payload Layer: State-Based Semantics . . . 30
5.3.1 Payload Semantics . . . 31
5.3.2 The `PayloadFramework` Module . . . 31
5.3.3 Payload Instantiation and Message Processing . . . 32
5.4 Backbone Layer: Causal Metadata and Replication . . . 33
5.4.1 Clock Management and Message Delivery . . . 35
5.4.2 Composition of State and Causality . . . 36
5.5 CRDT Adapters and Instantiation . . . 36
6 Verifying CRDT Implementations . . . 37
6.1 Introduction . . . 37
6.2 PN-Counter: A Showcase of Verified Properties . . . 38
6.2.1 Specification and Properties . . . 38
6.3 LWW Register: Deterministic Concurrent Write Resolution via Vector Clocks . . . 42
6.3.1 The Challenge . . . 42
6.3.2 State and Operations . . . 42
6.4 OR-Set: Composing CRDT State with Vector Clocks . . . 44
6.4.1 The Challenge: Re-addition Without Causality . . . 45
6.4.2 State Representation and Extended Merge . . . 45
6.5 The Adapter Pattern in Practice . . . 47
6.5.1 Adapter Structure . . . 47
6.5.2 Inheriting Properties via Adapters . . . 48
6.5.3 Executable Demonstration . . . 49
6.6 What We Verify: Summary . . . 50
7 Validation . . . 53
7.1 Proof Verification . . . 53
7.1.1 Coverage and Status . . . 53
7.2 Connection to Convergence . . . 53
7.3 How Verification Works . . . 54
7.4 Implications . . . 55
8 Discussion . . . 57
8.1 Framework Evaluation . . . 57
8.2 Future Directions . . . 57
9 Conclusions and Future Work . . . 59
Bibliography . . . 61

1 Introduction

Distributed systems replicate data across nodes to allow performance and availability. However, keeping all replicas consistent when dealing with concurrency, delays, and network partitions is a significant challenge. Traditional solutions that enforce strong consistency usually rely on consensus protocols and global coordination, which can be expensive and unstable under failure scenarios. As an alternative solution, *eventual consistency* is proposed. In this case updates may be applied independently at different replicas and reunified later in the process.

Conflict-Free Replicated Data Types (CRDTs) were defined to formalize this idea. CRDTs are data structures that allow updating concurrently and can be applied without coordination but still guarantee that all the replicas eventually reach the same state, as long as they receive the same set of updates. This property, known as *Strong Eventual Consistency* (SEC) is achieved through the design of asynchronous update and merge operations which have to satisfy algebraic conditions including commutativity, associativity, idempotence, and monotonicity [13].

Designing such data types is not always straightforward. Even small design choices, for instance how to handle removals in sets, can introduce issues that may conduct to divergence or break the intended semantics. Formal verification provides a solution to avoid behavioral errors by proving that a CRDTs satisfy properties like convergence and commutativity.

Prior work has demonstrated how to specify and verify CRDTs in theorem provers like Isabelle/HOL [18], establishing rigorous correctness guarantees for individual CRDT implementations. This thesis explores a structured approach by suggesting a more general purpose framework in the Athena proof assistant for specifying and verifying multiples types of CRDTs. The goal is to reduce the effort of building dedicated frameworks to verify each independent implementation of CRDTs by identifying common patterns in their design and behavior, using these patterns in reusable components. Our proposed framework aims to support the definition of state-based

CRDTs and helps express their semantics in a modular way. It also provides the structure to reason about properties like convergence and idempotence.

The contribution is not a complete or fully automated system but a set of structures that make it easier to work with CRDTs in Athena. In particular, it is aimed at situations where developers want to experiment with CRDT variants and need a way to express their behavior precisely and check key correctness properties. While this framework is limited in scope and not intended to cover every possible CRDT design; it offers a modest step toward making formal verification more practical in this domain.

We validate the framework by specifying and verifying six different CRDTs spanning both state-based and causal designs: GCounter, GSet, PN-Counter, Two-Phase Set, OR-Set, and LWW-Register. Verification proceeds by proving that each CRDT's merge and update operations satisfy five semilattice axioms (commutativity, associativity, idempotence, identity, and monotonicity). These axioms overall imply Strong Eventual Consistency. We show that our framework enables modular proof reuse: convergence is proven once at the abstract level and inherited by all CRDT instances. Validation includes demonstrations showing replicas converging under concurrent updates, causality tests verifying behavior under happened-before ordering, and tests confirming support to message reordering.

The thesis is organized as follows. Chapter 2 surveys existing CRDT verification approaches and identifies the gap our framework addresses. Chapter 3 introduces the Athena proof assistant, explaining its deductive reasoning capabilities and proof construction methodology that enable our verification framework. Chapter 4 develops the mathematical foundations: join-semilattice theory and abstract CRDT axioms. Chapter 5 presents the framework architecture, integrating payload semantics (state-based CRDTs), backbone replication (causal delivery), and the adapter pattern for instantiation. Chapter 6 showcases examples of verified CRDTs using the framework, demonstrating how adapters connect specifications to theory. Chapter 7 validates that all axiom proofs are mechanically type-checked by Athena, ensuring that convergence guarantees are not assumptions but proven facts. Chapter 8 evaluates the approach critically, discussing strengths and limitations. Finally, Chapter 9 summarizes contributions and propose future directions.

2 Related work

This thesis focuses on state-based CRDTs, where replicas exchange full state and use lattice-theoretic merge operations. However, we also discuss actor-based (operation-based) approaches because: (1) they provide complementary verification techniques that inform our design, (2) recent work has shown unified specifications can bridge both paradigms [11], and (3) understanding the operational perspective clarifies the distinction between commutativity requirements in state-based vs. operation-based designs. Our framework targets state-based CRDTs, but the literature review encompasses both approaches to contextualize our contributions.

This chapter reviews foundational literature on Conflict-Free Replicated Data Types (CRDTs). We begin by outlining the core principles of CRDTs and their role in achieving eventual consistency. The main focus is on formal specification and verification of these data structures. We survey various approaches including manual proof-checking, automated verification engines, and type-system-based methods. By surveying the state-of-the-art, this chapter establishes the context for our work and identifies research gaps that this thesis aims to address.

2.1 Conflict-Free Replicated Data Types and eventual consistency

To provide context for CRDT verification, we start by reviewing the main ideas related to CRDTs and their role in Eventually Consistent systems. Replication is a typical strategy for achieving high availability and fault tolerance in distributed systems, however maintaining the consistent aspect of replicated data can be expensive. Traditional approaches that require strong consistency rely on global consensus which may create performance conflicts with availability

under network partitions [13]. The CAP theorem states that a distributed system can not guarantee availability, partition tolerance and strong consistency at the same time, thus systems often weaken consistency in favour of availability and partition tolerance [18].

Shapiro et al. formalised this trade-off by introducing the model of *Strong Eventual Consistency* (SEC). In the context of an eventually consistent system, updates when executed at one replica are propagated to other replicas asynchronously, possibly in different orders: once all updates have been distributed, all replicas must converge to the same state [13]. Under SEC two properties are required: (1) *eventual delivery* which means every update that is delivered to one specific replica eventually is delivered to all replicas, and (2) *convergence*, meaning that replicas that have delivered the same updates eventually reach equivalent states [10]. CRDTs are a class of replicated data structures aimed to satisfy SEC by construction. They mix conflict resolution into the data type itself allowing updates in a concurrent state to be applied locally and merged deterministically without any coordination. Early work observed that both state-based and operation-based CRDTs can be specified by algebraic conditions: in *state-based* CRDTs, the state space is a join-semilattice and the merge operation computes the least upper bound requiring commutativity, associativity and idempotence [18, 13], in *operation-based* CRDTs, updates (operations) are broadcast in causal order and they have to commute so that replicas that apply the same set of operations reach the same state [10].

Designing CRDTs that guarantee SEC is non trivial. One of the most simple examples such as grow-only counter can be specified using per replica counters and merged by taking component-wise maxima [18]. Some other more complex data types such as sets that support both addition and removal require semantics designed to avoid anomalies like resurrecting deleted elements.

2.1.1 Classification of CRDTs: State-based and Operation-based Approaches

CRDTs are classified into two families based on what is replicated: *state-based* and *operation-based* designs. In a *state-based* CRDT, each replica maintains local state and periodically sends it to the rest of the replicas. Merging received state with local state requires a deterministic *merge* function. SEC is guaranteed whenever the state space forms a join-semilattice and the merge

function computes the least upper bound, requiring that merge be commutative, associative, and idempotent, and that local updates be monotone (moving state upward in the lattice) [18, 13]. In an *operation-based* CRDT, replicas broadcast *operations* (or *effectors*) instead of full state. Each update splits into a generator and an effector. SEC requires that effector broadcast be always causal, reliable, and that concurrent effectors commute [13, 10]. Both approaches rely on algebraic properties to guarantee convergence, but they differ fundamentally: state-based CRDTs assume replicas can exchange full state and require monotonicity at the data-type level, while operation-based CRDTs assume ordered delivery and require operational commutativity.

2.2 Formal specification and verification frameworks

Early verification efforts were conducted to provide machine-checked proofs that CRDT designs actually satisfy SEC. Zeller, Bieniusa and Poetzsch-Heffter developed an Isabelle/HOL framework for specifying and verifying CRDTs [18]. Their framework models an specific number of replicas. Their model abstracts over payloads, update operations and merge functions, representing each replica's state and history with associated metadata and version vectors. This framework enabled the first machine checked convergence proofs for CRDTs.

While the Isabelle framework is practical, it requires significant manual effort. To reduce this detail, Nagar and Jagannathan formulated a parameterised verification apporach that obtains convergence conditions from a given consistency model [9]. Their engine shows that not all operations must commute to guarantee convergence and produces automated proofs for CRDTs under various weak consistency policies [9].

Separation logic has been transformed to verify CRDT implementations. Nieto et al. introduced OpLib and RcbLib libraries that allow modular verification for operation-based CRDTs using Aneris distributed separation logic. These libraries abstract away low-level concurrency concerns and provide mechanised Coq proofs for a collection of replicated data structures [10]. The approach was later extended to state-based CRDTs because states grow monotonically to include all updates, unified specifications can be written without distinguishing between state and operation based designs [11].

Recent research also uses type systems, domain-specific languages and program synthesis to

guarantee CRDT correctness by construction. Propel is a Scala-embedded language whose type system reasons about algebraic properties such as commutativity, associativity and idempotence, ensuring that merge functions satisfy these laws [17]. VeriFx takes a complementary approach by integrating verification into a high-level language: programmers implement replicated data types and specify correctness properties, and the language automatically proves those properties and converts verified implementations to mainstream languages [12]. Liang and Feng observed that strong eventual consistency does not ensure functional correctness and proposed Abstract Converging Consistency (ACC), which augments CRDT specifications with abstract atomic operations, enabling modular verification of both implementations and clients [7]. Katara automatically generates verified CRDTs from sequential data types by encoding correctness conditions into SMT logic and searching for suitable semilattice representations [6].

2.3 Byzantine fault tolerance and CRDTs

Kleppmann showed that augmenting CRDT algorithms with signatures, hashes and vector clocks can tolerate an arbitrary number of Byzantine replicas while preserving strong eventual consistency [5]. More recent work identifies CRDT classes that prevent equivocation; for example, Jacob, Bayreuther and Hartenstein observed that the Matrix Event Graph stores updates in a hash-based directed acyclic graph and tolerates equivocation without additional coordination [4]. Such results highlight both the feasibility and the limits of building Byzantine-tolerant CRDTs.

Tschudin proposed a “connectionless” grow-only set CRDT that operates over lossy, reorderable channels without the necessity of any persistent connections between nodes. This CRDT uses a single message type and incremental updates to spread state, making it suitable for applications such as peer-to-peer compression dictionaries where maintaining per-node connections is unfeasible [15]. Such innovations widen the scope of the CRDT paradigm to networks where communication can be intermittent. Additional mechanisms may be necessary to handle message loss and ensure eventual convergence.

2.4 Verification in actor and concurrency frameworks

Musser and Varela developed a structured reasoning framework for actor systems using the Athena proof language. They established abstract theorems about fairness, persistence, message delivery and progress that hold for any actor system, independently of implementation details [8]. Their work demonstrates that high-level algebraic reasoning can give general properties such as guaranteed message delivery and monotonicity of actor states [8]. These results motivate the use of high-level proof environments when reasoning about replicated data structures.

Formalisation of concurrent languages has also lead to the development of modular semantics used in many proof frameworks. Bereczky, Horpácsi and Thompson presented a machine-checked formalisation of concurrent Core Erlang using modular frame stack semantics [1]. They proved properties such as determinism, confluence and program equivalence [1] providing a foundation for verifying concurrency properties of actor-based programming languages.

The actor model has also inspired languages for dynamic and open systems. For instance, SALSA compiles to Java and provides primitives for actor creation, asynchronous messaging, state updates, continuations and universal naming to coordinate distributed actors [16]. It emphasises disconected message passing and mobility, repeating with the asynchronous update and merge semantics of CRDTs and illustrate how actor based languages can support reasoning about distributed computations.

2.4.1 Program Synthesis and Automated CRDT Generation

Katara (2022) introduced a novel approach to synthesizing CRDTs using verified lifting, allowing developers to generate CRDT implementations automatically from sequential data types. This synthesis approach leverages lightweight ordering constraints and inductive invariants enabling the automated generation and verification of new CRDTs [6]. This automation addresses the inherent difficulty in designing CRDTs manually and offers a way to scale CRDTs to new applications.

2.4.2 Comparative Analysis: Defining Grammar, Rule Inference, and Formal Verification

In order to provide a clearer understanding of the landscape of CRDT research, we summarize the state-of-the-art in Table 2.1 that highlights the following dimensions for each paper: (1) Definition of CRDT grammar, (2) Inference of rules or operations, (3) Implementation in programming languages, (4) Formal verification of properties, and (5) Byzantine Fault Tolerance (BFT).

Table 2.1: Comparison of CRDT research proposals

Paper/Research	**Grammar**	**Rule**	**Language**	**Verification**	**BFT**
Zeller et al. (2014)	Yes	Yes	Isabelle/HOL	Yes	No
Katara (2022)	Yes	Yes	Automatic (synthesis)	Yes (automated)	No
Syncordian (Sanabria & Cardozo, 2022)	Yes	Yes	Elixir	Yes (Athena)	Yes
Kleppmann (2022)	No	Yes	Not specified	No	Yes
Nagar & Jagannathan (2019)	Yes	Yes	Not specified	Yes (automated)	No
Sanjuan et al. (2020, Merkle-CRDTs)	No	Yes	IPFS	No	No
Liang & Feng (2021)	Yes	Yes	Not specified	Yes (modular)	No

2.4.3 Discussion

From the comparative analysis, it becomes evident that formal verification remains one of the most significant challenges in CRDT development. Early frameworks like Isabelle/HOL provide rigorous correctness guarantees but require expert involvement. Automated approaches such as parameterised verification [9], separation logic libraries [10], type systems [17] and domain-specific languages [12] lower the burden of proof but they are often tailored to specific models. Recent efforts to provide abstractions for functional correctness such as ACC [7], and equivocation tolerance in Byzantine environments [4] suggest that when CRDTs are used as building blocks in complex systems richer correctness specifications are required.

While many researchers have provided grammars and rules for CRDT operations, fewer have focused on automating the verification process. Tools like Katara and the verification engine

developed by Nagar and Jagannathan represent important advances in reducing the complexity of designing and verifying CRDTs [6, 9]. Work on Byzantine fault tolerance shows that many CRDT algorithms can be extended to handle malicious replicas through cryptographic techniques [5] but systematic verification of such extensions remains scarce. While Byzantine Fault Tolerance is gaining importance with the rise of decentralized systems, only a few CRDT implementations focus on this aspect. Kleppmann's work on Byzantine Fault Tolerance provide significant contributions in this regard, but much remains to be explored in ensuring that CRDTs can function reliably in adverse environments [5].

2.4.4 Conclusion

The research on CRDTs has advanced significantly over the last decade, particularly in defining the grammar and rules for these data structures and verifying their convergence and consistency. However, challenges like automating the verification process and extending CRDTs to tolerate Byzantine faults remain. As distributed systems continue to evolve, the ability to design, implement, and verify CRDTs will be crucial for building scalable, resilient systems.

3 Athena in a Nutshell

This chapter introduces Athena, the declarative proof language used over this thesis for formalizing CRDT specifications. Athena [3] is a multi-paradigm language that combines automated reasoning with user-guided proof construction, making it well-suited for theorem proving about algebraic structures like semilattices.

We present the essential Athena constructs through practical examples, building from basic declarations to complete proofs. This prepares readers to understand the formal specifications without requiring prior Athena experience.

3.1 Why Athena for CRDT Verification?

CRDTs rely on algebraic properties (commutativity, associativity, idempotence) that must hold for all states and operations. Athena provides:

- **Equational reasoning**: Natural expression properties through equality chains
- **Modularity**: Theory hierarchies that factor common proofs
- **Automation**: Built-in inference methods reduce proof load.

3.2 Basic Constructs

3.2.1 Modules and Namespaces

Modules encapsulate related definitions and theorems. Each module creates a namespace that enables a hierarchical organization of proofs.:

```
module GCounter {
  # Declarations visible within GCounter module
  declare merge: [N N] -> N

  module Theory {
    # Nested module for theorems
    conclude commutative := (forall ?n1:N ?n2:N .
      (merge ?n1 ?n2) = (merge ?n2 ?n1))}
}

(!chain [(GCounter.merge x y)
       = (GCounter.merge y x)  [GCounter.Theory.commutative]])
```

Snippet 3.1: Module definition with nested namespace

The open directive imports module contents into the current namespace, reducing verbosity when working extensively with a module's statements.

3.2.2 Sorts and Declarations

Athena is multi-sorted: every term has a sort (type), and operations declare their input and output sorts. The declare keyword introduces uninterpreted function symbols:

```
# Define custom sorts
define-sort State := (Pair N N)  # Product type
datatype Value := (int N) | (str String)  # Sum type

# Declare operations with sort signatures
declare initial: [] -> State        # Nullary (constant)
declare update: [State Value] -> State  # Binary function
declare merge: [State State] -> State   # Binary operation

define [s s1 s2] := [?s:State ?s1:State ?s2:State]
```

Snippet 3.2: Sort declarations and function signatures

Pattern variables (prefixed with ?) range over their declared sorts. The expression define [s ...] := [?s:State ...] reduces repetition in axioms and theorems.

3.2.3 Assertions: Stating Axioms

The assert directive introduces axioms/propositions assumed without proof. The assert* variant allows multiple axioms in a list:

```
# Single assertion
assert merge-commutative :=
  (forall ?s1:State ?s2:State . (merge s1 s2) = (merge s2 s1))
# Multiple related axioms
assert* merge-axioms := [
  (forall ?s1 ?s2 . (merge s1 s2) = (merge s2 s1))  # Commutativity
  (forall ?s . (merge s s) = s)                      # Idempotence]
```

Snippet 3.3: Axiomatic specification of merge

Axioms form the foundation of a specification.

3.2.4 Proofs: The `conclude` Construct

While assert states axioms, conclude proves theorems. A proof provides a *deduction* that justifies the conclusion:

```
conclude merge-symmetric :=
  (forall ?s1:State ?s2:State .
    (merge s1 s2) = (merge s2 s1) ==> (merge s2 s1) = (merge s1 s2))
  pick-any s1:State s2:State
    assume hyp := ((merge s1 s2) = (merge s2 s1))
      (!sym hyp)  # Symmetry of equality
```

Snippet 3.4: Basic proof structure

The proof structure follows the logical form:

1. pick−any introduces universally quantified variables
2. assume binds the hypothesis of an implication
3. Inference methods (prefixed with !) construct the deduction

3.3 Proof Methods

Athena provides primitive inference rules as well as high-level proof methods. Methods automate common reasoning patterns.

3.3.1 Equational Reasoning with `!chain`

The !chain method builds a chain of equalities, where each step is justified by an axiom or theorem:

```
conclude merge-associative-demo :=
  (forall ?s1:State ?s2:State .
    (merge (merge s1 s2) s2) = (merge s1 s2))
  pick-any s1:State s2:State
    (!chain [(merge (merge s1 s2) s2)
           = (merge s1 (merge s2 s2))  [merge-associative]
           = (merge s1 s2)             [merge-idempotent]])
```

Snippet 3.5: Equational chain proof

Each line in the chain states an equality, and the term in brackets names the justification. This style emulates standard mathematical practice and makes proofs highly readable.

3.3.2 Implication Chain with `!chain->`

For proving implications, !chain−> sequences modus ponens steps:

```
# Given: (P ==> Q) and (Q ==> R) axioms
conclude transitive-implication := (P ==> R)
  (!chain-> [P ==> Q  [axiom-1]
             ==> R  [axiom-2]])
```

Snippet 3.6: Implication chain

This is particularly useful for monotonicity proofs, where we show that one property implies another through a sequence of intermediate steps.

3.3.3 Universal Instantiation with `!uspec`

When an axiom or theorem is universally quantified, !uspec specializes it to concrete terms:

```
# Given axiom: (forall ?s1 ?s2 . (merge s1 s2) = (merge s2 s1))
assert merge-commutative := (forall ?s1 ?s2 . ...)

# Specialize to specific states
define instance := (!uspec merge-commutative s1 s2)
# Now instance is: (merge s1 s2) = (merge s2 s1)
```

Snippet 3.7: Universal specialization

Athena offers both !chain and !uspec as formal methods. The following are equivalent:

```
# Manual
define step1 := (!uspec merge-commutative s1 s2)
define result := (!chain [(merge s1 s2) = (merge s2 s1) [step1]])

# Automatic
define result := (!chain [(merge s1 s2) = (merge s2 s1) [merge-
    commutative]])
```

Snippet 3.8: Automatic vs manual instantiation

3.3.4 Case Analysis with `!cases`

For datatype definitions, !cases performs structural induction or case splits:

```
datatype Update := (increment) | (decrement)

conclude update-total :=
  (forall ?u:Update . (u = increment) | (u = decrement))
  pick-any u:Update
    (!cases u {
      (increment) => (!left (!reflex increment))
      (decrement) => (!right (!reflex decrement)) })
```

Snippet 3.9: Proof by cases over a datatype

3.4 Theories and Adapters

Athena's *theory mechanism* enables modular proofs. A theory packages axioms and derived theorems, which can then be instantiated for multiple concrete types.

3.4.1 Defining a Theory

A theory is created with make−theory, specifying axioms and theorems:

```
module SemilatticeTheory {
  declare join: (T) [T T] -> T
  assert commutative := (forall ?x ?y . (x join y) = (y join x))
  assert associative := (forall ?x ?y ?z .
    ((x join y) join z) = (x join (y join z)))
  assert idempotent := (forall ?x . (x join x) = x)

  # Package into a theory
  define theory := (make-theory [] [commutative associative idempotent])
}
```

Snippet 3.10: Abstract semilattice theory

3.4.2 Instantiating via Adapters

To apply the theory to a concrete CRDT, we create an *adapter* that maps the abstract operations to concrete ones:

```
module GCounterAdapter {
  # Map abstract 'join' to concrete 'gc-merge'
  define merge := gc-merge

  # Prove adapter satisfies theory axioms
  conclude commutative :=
    (forall ?s1:N ?s2:N . (gc-merge s1 s2) = (gc-merge s2 s1))
    pick-any s1:N s2:N
      (!chain [(gc-merge s1 s2)
             = (N.max s1 s2)          [gc-merge-definition]
             = (N.max s2 s1)          [N.Max.commutative]
             = (gc-merge s2 s1)       [gc-merge-definition]])

  # Similar for associative, idempotent...
  # Now all theorems from SemilatticeTheory hold for gc-merge
}
```

Snippet 3.11: CRDT adapter inheriting semilattice properties

This pattern eliminates redundant proofs: once we've proven that N.max is a semilattice, any CRDT using max-based merge inherits those properties.

3.5 Reading Athena Code

When encountering Athena code in later chapters, readers should:

1. **Identify the module structure**: What operations are declared? What sorts are involved?
2. **Locate the axioms**: Which properties are asserted vs. proven?
3. **Follow the proof chains**: Equational steps mirror standard algebraic reasoning
4. **Check the justifications**: Each step cites an axiom, theorem, or inference rule

The formal specifications serve two purposes:

- **Documentation**: Precise semantics for CRDT operations
- **Verification**: Machine-checked proofs of convergence properties

3.6 Tools and Workflow

Athena proofs are developed interactively. A typical workflow:

1. Write the specification (sorts, declarations, axioms)
2. State the theorem to prove with conclude
3. Build the proof incrementally, checking each step
4. Refactor common patterns into reusable methods

The Athena interpreter provides immediate feedback on type errors, failed deductions, and missing justifications, enabling rapid iteration.

With these fundamentals in hand, readers are prepared to understand the formal CRDT specifications in Chapter 4 and the framework implementations in Chapter 5. The subsequent chapters assume familiarity with the definitions presented here, focusing on the algebraic structures and convergence properties rather than Athena syntax.

4 Theoretical Foundations

State-based Conflict-Free Replicated Data Types rely on join-semilattice theory. A join-semilattice is an algebraic structure in which any two elements can be combined via a *join* operation that is commutative, associative and idempotent. These three properties ensure that replicas merging their states in different orders will converge to the same result.

This chapter develops the theory behind our framework. We begin by defining join-semilattices and explaining how their algebraic properties guarantee convergence. We then present the abstract CRDT specification consisting of five axioms that, when satisfied, guarantee Strong Eventual Consistency (SEC).

Finally, we show how concrete CRDTs such as GCounter, GSet and PN-Counter instantiate this theory and inherit convergence guarantees through a *theory/adapter* proof pattern.

4.1 Semilattices

A *join-semilattice* is a set equipped with a binary operation satisfying three axioms. Formally:

Definition 4.1 (Join-Semilattice)**.** *A join-semilattice is a structure* $(S, \sqcup)$ *where* S *is a set and* $\sqcup : S \times S \to S$ *is a binary operation satisfying:*

1. ***Commutativity*** $\forall x, y \in S.\ x \sqcup y = y \sqcup x$
2. ***Associativity*** $\forall x, y, z \in S.\ (x \sqcup y) \sqcup z = x \sqcup (y \sqcup z)$
3. ***Idempotence*** $\forall x \in S.\ x \sqcup x = x$

These three properties are fundamental to CRDT convergence. Each property plays a specific role:

- **Commutativity** ensures the order in which two states are merged does not affect the result. If replica A merges with replica B, and replica B merges with replica A then they produce the same state.
- **Associativity** ensures when merging multiple states, the way they are grouped does not matter. Replicas can merge states in any order and produce the same result.
- **Idempotence** ensures merging a state with itself produces the same state. This property is crucial for handling duplicate messages or repeated merge operations.

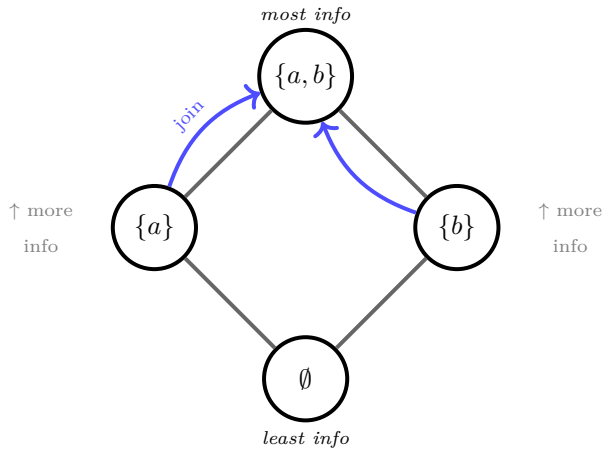


Figure 4.1: Hasse diagram of the join-semilattice $(\mathcal{P}(\{a,b\}), \cup)$. The power set of a two-element set $\{a,b\}$ is ordered by subset inclusion ($\subseteq$), forming a lattice. The join operation is set union ($\cup$): for example, $\{a\} \sqcup \{b\} = \{a,b\}$ is shown as the blue arrows (join). Elements higher in the diagram represent states with more information. The join of any two elements is their least upper bound.

The join operation induces a partial order on S defined by:

$$x \leq y \iff x \sqcup y = y.$$

Intuitively, $x \leq y$ means that y contains at least as much information as x. The join operation computes the least upper bound of two elements with respect to this order. Figure 4.1 illustrates

a simple join-semilattice as a Hasse diagram, where edges represent the covering relation and the join of two elements is their least common ancestor.

Partial order is crucial to guarantee CRDT convergence. Updates must be *monotonic*, meaning they only move states upward in the lattice. Once a replica has reached a certain state, subsequent merges can only increase information, never decrease it.

Semilattice theory in Athena

We encode the semilattice axioms in Athena as an abstract module that declares the join operation and asserts the three axioms. The module is polymorphic in the element type, allowing it to be instantiated for different data types. Listing 4.1 shows the core module definition:

```
module SemilatticeTheory {
  declare join: (T) [T T] -> T
  declare bot: (T) [] -> T
  define [x y z] := [?x ?y ?z]

  assert* Commutative := ((x join y) = (y join x))
  assert* Associative := (((x join y) join z) = (x join (y join z)))
  assert* Idempotent := ((x join x) = x)
  assert* LeftIdentity := ((bot join x) = x)
  assert* RightIdentity := ((x join bot) = x)

  define theory := (make-theory [] [Commutative Associative Idempotent
      LeftIdentity RightIdentity])}
```

Snippet 4.1: SemilatticeTheory: Abstract semilattice axioms

The module declares join as a polymorphic binary operation and bot as the bottom element (identity). The five assertions capture the semilattice axioms plus identity laws. We also define derived properties such as absorption and monotonicity that follow from the axioms. These are proven once in the abstract theory and inherited by all concrete instantiations. Listing 4.2

extends this theory with derived properties.

```
extend-module SemilatticeTheory {
  define absorption :=
    (forall x y . (x join y) = y ==> (y join x) = y)
  define proofs :=
    method (theorem adapt)
      let {[get prove chain] := (proof-tools adapt theory);
           [join bot] := (adapt [join bot])}
        match theorem {
          (val-of absorption) =>
            conclude (adapt theorem)
            pick-any x y
              assume hyp := ((x join y) = y)
                (!chain [(y join x) = (x join y) [Commutative]
                                    = y          [hyp]])}
  (add-theorems theory |{[absorption] := proofs}|)
}
```

Snippet 4.2: Extending SemilatticeTheory with derived properties

The !chain construct builds a chain of equalities justified by the axioms. Once proven, these theorems are added to the theory and can be used when verifying concrete CRDTs.

4.2 State-based CRDT model

Building on the semilattice foundation, we define an abstract state-based CRDT as a structure with five core components and corresponding axioms that guarantee convergence.

Definition 4.2 (Abstract CRDT)**.** *A CRDT is a five-tuple* $\langle S, V, \mathit{initial}, \mathit{update}, \mathit{merge}\rangle$ *where:*

- S *is the state domain (elements of the semilattice)*
- V *is the update domain (operations that modify state)*

- *initial* : $() \to S$ *returns the bottom element as State* S
- *update* : $S \times V \to S$ *applies an update* V *to a state* S
- *merge* : $S \times S \to S$ *is the semilattice join operation*

satisfying the five axioms listed below.

Convergence axioms

The abstract specification consists of five axioms that together guarantee convergence. For all states $s, s_1, s_2, s_3 \in S$ and updates $v \in V$:

A1. Merge Commutativity: $merge(s_1, s_2) = merge(s_2, s_1)$.

A2. Merge Associativity: $merge(merge(s_1, s_2), s_3) = merge(s_1, merge(s_2, s_3))$.

A3. Merge Idempotence: $merge(s, s) = s$.

A4. Initial Identity: $merge(initial(), s) = s$.

A5. Update Monotonicity: $merge(s, update(s, v)) = update(s, v)$.

This axiom ensures *progress*: once a replica applies an update, that update remains visible after merging with concurrent states. This guarantees no data loss during synchronization.

Axioms in Athena

The five axioms are encoded in Athena as assertions within a CRDT specification module. Listing 4.3 shows how these axioms appear in a concrete implementation (simplified GCounter):

```
1 module GCounter {
2   assert merge-commutative:= (forall n1 n2 .(merge n1 n2)=(merge n2 n1))
3   assert merge-associative:=
4     (forall n1 n2 n3 .(merge (merge n1 n2) n3)=(merge n1 (merge n2 n3)))
5   assert merge-idempotent:= (forall n . (merge n n) = n)
6   assert merge-left-identity:= (forall n . (merge initial n) = n)
7   assert update-monotonic:=
8     (forall n v .(merge n (update n v)) = (update n v))}
```

Snippet 4.3: The five CRDT axioms encoded in Athena

These axioms together guarantee convergence: commutativity and associativity ensure merge order independence, idempotence handles duplicate messages, identity anchors the lattice, and monotonicity preserves updates across replicas.

Strong Eventual Consistency

Strong Eventual Consistency (SEC) is a correctness criterion for replicated data types introduced by Shapiro *et al.* [13]. It consists of two properties:

Definition 4.3 (Strong Eventual Consistency)**.** *A replicated data type satisfies SEC if it guarantees:*

1. ***Eventual Delivery**: An update delivered at one replica is eventually delivered to all replicas.*
2. ***Convergence**: Replicas that have delivered the same set of updates have equivalent states.*

Eventual delivery is a liveness property that depends on the underlying network and delivery mechanism; it is typically assumed rather than proven. Convergence, on the other hand, is a safety property that can be proven from the CRDT axioms. The key insight is that the semilattice structure ensures deterministic convergence.

Convergence theorem

The central theorem in CRDT theory is the Strong Eventual Consistency Theorem, which establishes that when a CRDT satisfies the five axioms above, all replicas that receive the same set of updates will converge to a unique state. This fundamental result was formalized by Shapiro *et al.* [13] and forms the theoretical foundation for all CRDT designs.

4.3 Concrete CRDT theories in Athena

Having established the abstract theory, we now explain how concrete CRDTs specifications instantiate the abstract theory of CRDT without having to prove again the five axioms from scratch. Athena's *theory/adapter* pattern enables modular verification: prove properties once

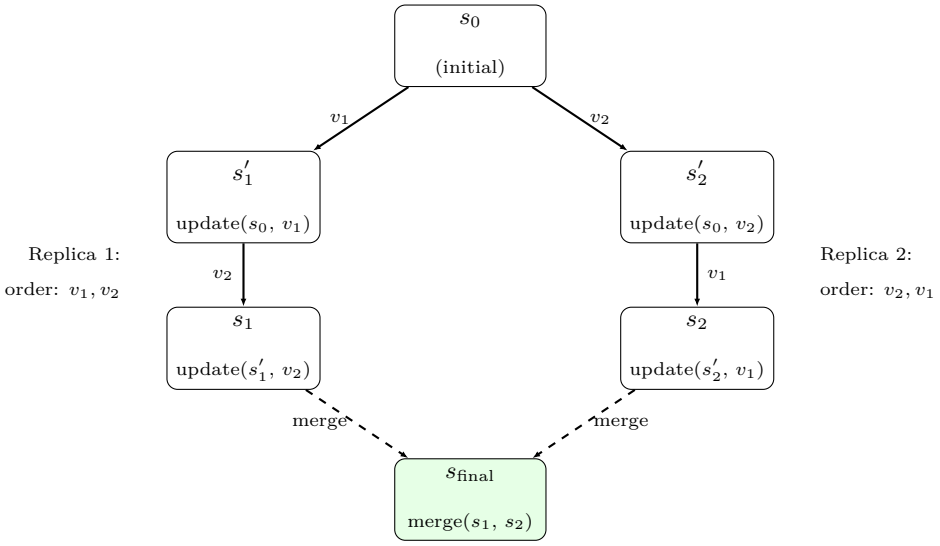


Figure 4.2: Strong Eventual Consistency from semilattice axioms. Two replicas start from the same state s_0 and apply updates v_1 and v_2 in different orders. By Axiom 5 (monotonicity), states increase at each step. By Axioms 1–3 (commutativity, associativity, idempotence), merging the final states yields the same result regardless of update order.

in a base theory, then create lightweight adapters for specific CRDTs that inherit those proofs automatically.

Many CRDTs share common algebraic structures:

- Counters use the max operation on natural numbers.
- Sets use the $\cup$ (union) operation on sets.
- Composite CRDTs often combine existing semilattices (e.g. via product constructions).

Instead of proving commutativity, associativity, and idempotence separately for each CRDT, we prove these properties once for the base operations (max, $\cup$, etc.) and then create a *renaming adapter* that maps the CRDT's operations to those initial operations.

For instance, a grow-only counter CRDT with merge operation defined pointwise by maximum:

$$\mathit{merge}(s_1, s_2) = \lambda i.\ \max(s_1(i), s_2(i)),$$

i.e. for each replica identifier i, the merged counter holds the maximum of the two replica values. If we have proven that max on $\mathbb{N}$ is commutative, associative, and idempotent, then the counter's merge inherits these properties directly by reduction to the max operation, without needing a new proof of ACI. Figure 4.3 illustrates the pattern architecture.

We apply this pattern systematically across our CRDT implementations:

- **Counters** (GCounter, PN-Counter) reuse the theory of max on $\mathbb{N}$.
- **Sets** (GSet, Two-Phase Set) reuse the theory of $\cup$ (set union) on power sets.
- **Causal CRDTs** (OR-Set, LWW-Register) compose a payload semilattice with a vector clock semilattice.

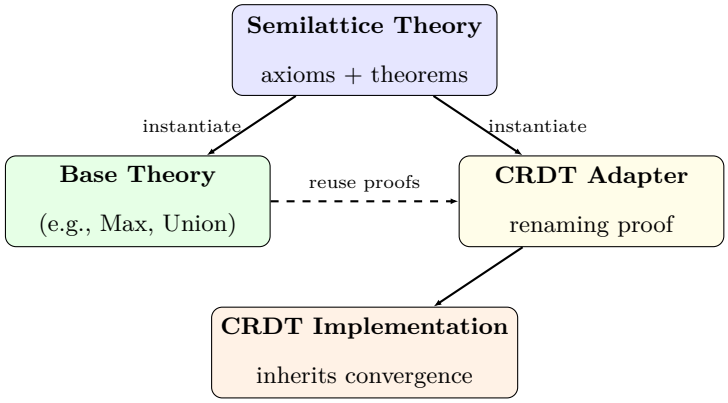


Figure 4.3: The theory/adapter pattern for modular CRDT verification. Abstract semilattice theorems are proven once at the theory layer and instantiated via base theories (e.g., max, $\cup$). CRDT adapters provide renaming maps that inherit all proven properties. This pattern reduces redundant proof effort across multiple CRDTs.

Each adapter proof is mostly mechanical, reducing the CRDT axioms to the base semilattice axioms, whereas a from-scratch proof would require repeating the reasoning about commutativity, associativity, idempotence, and monotonicity each time.

the theory layer also supports building semilattices from smaller ones:

- **Product semilattices**: If $(S_1, \sqcup_1)$ and $(S_2, \sqcup_2)$ are semilattices, then their product $(S_1 \times$

$S_2, \sqcup$) defined by

$$(a, b) \sqcup (c, d) = (a \sqcup_1 c,\ b \sqcup_2 d)$$

is also a semilattice. We use this principle to construct composite CRDTs like the PN-Counter (which combines a positive and a negative counter). By proving the product construction correct once, we can reuse it for any combination of semilattices.

- **Causal overlays**: For CRDTs requiring causality tracking, we layer vector clocks over payload semilattices. Section 5.4 of the framework provides this composition: the vector clock timestamps also form a semilattice (under pointwise max), which is combined with the payload's semilattice. This ensures that causal metadata and payload state both grow monotonically and can be merged without conflict.

The six concrete CRDT implementations verified in this work follow this approach and are presented in detail in Chapter 6.

In summary, the algebraic laws of semilattices and the five CRDT axioms provide a foundation for eventual consistency. By coding these laws in Athena's theory modules, we can build replicated data types through extensions and adapters, with convergence proofs largely automated. In the next chapter, we build on this foundation to develop the Payload Framework, which provides the executable semantics for state-based CRDT replication.

5 Framework Architecture

5.1 Overview of the Layered Architecture

The verification framework is organized as a stack of four layers, as illustrated in Figure 5.1: the Theory layer as the base, the Payload layer above it, the Backbone layer next, and concrete CRDT implementations at the top. Each layer builds on the abstractions of the layer below it, maintaining a clear separation of purpose that isolates algebraic correctness, state semantics, replication methods and specific logic into separate modules. This modularity enables extensibility and facilitates reasoning: each layer can be specified and verified in isolation, then

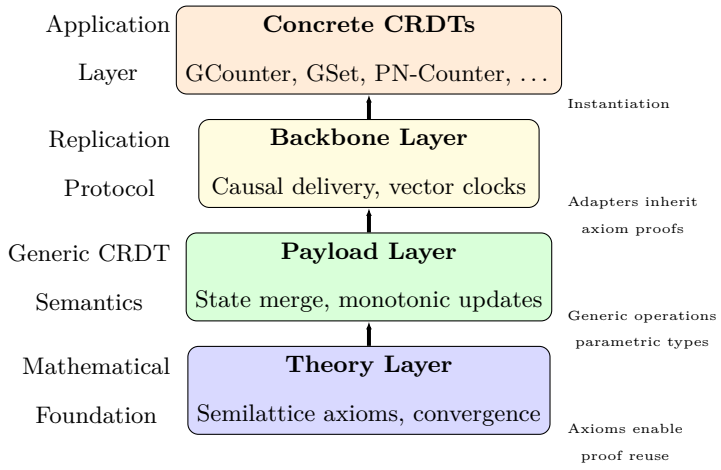


Figure 5.1: Layered framework architecture showing composition of theory, payload, backbone, and concrete implementations. Each layer inherits correctness guarantees from the layer below through the theory/adapter pattern. Axioms proven in the Theory layer are reused by all upper layers.

composed to obtain full end-to-end guarantees. Section 5.2 covers the foundational theory module, Section 5.3 describes the payload semantics, Section 5.4 discusses the backbone for causal consistency, and Section 5.5 explains how concrete CRDT implementations are connected using adapters.

5.2 Theory Layer: `SemilatticeTheory` and Abstract CRDT

The foundation of the framework is a formal semilattice theory (developed in Chapter 4) that captures the algebraic conditions for convergence using five axioms: commutativity, associativity, and idempotence of merge; existence of an identity element; and monotonicity of updates. These axioms guarantee Strong Eventual Consistency (SEC). The Theory layer provides a mathematical foundation that all CRDT instances share: key convergence theorems are proven once at the abstract level, and each concrete CRDT can inherit these theorems by showing that its state and operations instantiate the Theory layer's axioms. This *theory/adapter pattern* means each CRDT needs only provide a small proof (via an adapter) that the theory's conditions hold, rather than reproving convergence from scratch.

The foundation also includes utilities that support the upper layers: library modules for finite maps (to represent mappings from replica identifiers to states or timestamps), multisets (to represent network message buffers), and a formalisation of vector clocks (finitely supported maps from replica IDs to counters).

5.3 Payload Layer: State-Based Semantics

The Payload layer implements the core semantics of state-based CRDTs on top of the abstract theory. This layer is supported by the `PayloadFramework` module which defines how CRDT states are maintained and how they evolve through message passing. The payload framework remains parametric over the actual CRDT state and update types: it assumes nothing about the concrete data structure beyond the semilattice properties ensured by the Theory layer. Instead, it provides a generic execution model that any semilattice-based CRDT can work with.

5.3.1 Payload Semantics

The Payload layer implements state-based CRDT semantics using the semilattice model from Chapter 4. CRDT states live in such lattices, and updates must be monotonic with respect to the lattice order, ensuring SEC. Refer to Chapter 4 for the formal development of semilattice axioms and convergence guarantees.

5.3.2 The `PayloadFramework` Module

Each CRDT specification declares state and operation types, defines merge, initial, and apply-update functions, and provides axioms for these operations. Listing 5.1 shows the GCounter specification as a representative example:

```
module GCounterSpec {
  define-sort GCState := N
  datatype GCValue := (increment N)

  declare gc-merge: [GCState GCState] -> GCState
  declare gc-initial: GCState
  declare gc-apply-update: [GCState GCValue] -> GCState

  assert* gc-merge-is-max := [((gc-merge ?x ?y) = (N.Max ?x ?y))]
  assert* gc-initial-def := [(gc-initial = zero)]
  assert* gc-apply-update-def :=
    [((gc-apply-update ?s (increment ?delta)) = (N.+ ?s ?delta))]
}
```

Snippet 5.1: GCounter specification showing CRDT pattern

The definition states that `merge` is a commutative, associative, idempotent operation with `leq` as the defined order. The axiom `update-monotone` requires that applying an update never decreases the state. Together, these conditions ensure that the state space forms a join-semilattice and that `apply` is monotonic. Shapiro *et al.* [13] prove that any state-based CRDT satisfying

these axioms is strongly eventually consistent.

Progress Property. A CRDT satisfies the *progress property* if local updates are never lost during merge operations. Formally:

$$merge(s, apply(s, v)) = apply(s, v)$$

This property ensures that once a replica applies an update, that update remains visible in all future merged states. Progress is a direct consequence of Axiom A5 (Update-Monotonic) and is critical when ensuring concurrent updates from different replicas converge without data loss. Three related monotonicity concepts are distinguished:

1. **Merge Monotonicity (Partial Order)**: States increase through merge: $s \sqsubseteq (s \sqcup t)$. This follows from the semilattice structure.

2. **Update Monotonicity (Axiom A5)**: Updates increase state: $s \sqsubseteq apply(s, v)$. Equivalently: $s \sqcup apply(s, v) = apply(s, v)$.

3. **Progress (Preservation through Merge)**: Updates survive merging: $merge(s, apply(s, v)) = apply(s, v)$. This ensures updates are not lost during synchronization.

Progress properties are proven explicitly for specific CRDTs where they are critical to correctness (e.g., PN-Counter's `merge-preserves-pos-update`, GSet's `merge-preserves-additions`). Merge monotonicity follows from the semilattice structure, while update monotonicity is established using CRDT-specific proofs that show updates move upward in the lattice order.

5.3.3 Payload Instantiation and Message Processing

The `Payload` module provides a wrapper for instantiating the abstract `PayloadFramework`. A payload instance stores the initial state, and two abstract operations `payload-step` and `payload-join` must be defined for that payload. A configuration contains a map from actor identifiers to their local states and a multiset of pending messages. The core operations are:

Message reduction. The `reduce` function models the delivery of a single message. If the message network is empty, the configuration is returned unchanged. Otherwise, one message is removed from the multiset, the payload is applied to the current state and the state map is updated. Listing 5.2 implements this operation in the Payload module:

```
define (reduce apply-fn cfg) :=
  match cfg {
    (kfg actors MSet.null) => cfg
  | (kfg actors (MSet.insert (msg id payload meta) rest)) =>
      match (map-lookup actors id) {
        NONE => (kfg actors rest)
      | (SOME current) =>
          (kfg (FMap.update (pair id (apply-fn current payload)) actors)
               rest)}
  }
```

Snippet 5.2: Message reduction in Payload module

The payload layer ensures that merges are always commutative and idempotent. Replicas that receive the same set of updates in any order will converge to the same state.

5.4 Backbone Layer: Causal Metadata and Replication

While the payload layer works for purely state-based CRDTs, many CRDT designs require tracking causal ordering. The Backbone layer extends the framework with a replication protocol using causal metadata via vector clocks.

A vector clock $C : Ide \to \mathbb{N}$ maps replica identifiers to counters. Two clocks are ordered $C \preceq D$ if $C(i) \leq D(i)$ for all i. The join operation $(C \sqcup D)(i) = \max(C(i), D(i))$ forms a semilattice.

Concretely, vector clocks are formalized in Athena as an inductive datatype with constructor axioms, as shown in Listing 5.3:

```
datatype Clock := clock-empty
                | (clock-update ActorId Timestamp Clock)

declare clock-get: [Clock ActorId] -> Timestamp
declare clock-join: [Clock Clock] -> Clock

assert* clock-join-axioms :=
  [(# Base case: joining empty clock gives second clock
    (clock-join clock-empty clock2) = clock2)

   (# Recursive case: update with max timestamp
    (clock-join (clock-update actor count1 rest) clock2) =
    (clock-update actor (N.Max count1 (clock-get clock2 actor))
                  (clock-join rest clock2)))]
```

Snippet 5.3: Vector clock definition: inductive datatype with operations

The `Backbone` module extends payload semantics with vector clock semantics. A configuration is equipped with a clock map in addition to the state map and message buffer. Listing 5.4 shows the configuration datatype:

```
datatype (Configuration State Value) :=
  (kfg astates:(FMap.Map Ide State)
       clocks:(FMap.Map Ide Clock)
       net:(MSet.MSet (Message Value Clock)))
```

Snippet 5.4: Configuration datatype for Backbone module

Listing 5.5 shows these accessor functions. The helper functions select the appropriate fields without exposing the complete structure internally.

```
define (get-cfg-states cfg) :=
  match cfg {
    (kfg states _ _) => states
  | _ => FMap.empty-map}
```

```
5
6  define (get-cfg-clocks cfg) :=
7    match cfg {
8      (kfg _ clks _) => clks
9    | _ => FMap.empty-map}
10
11 define (get-cfg-net cfg) :=
12   match cfg {
13     (kfg _ _ network) => network
14   | _ => MSet.null}
```

Snippet 5.5: Backbone configuration accessors: encapsulating internal structure

5.4.1 Clock Management and Message Delivery

When a replica performs a local update, its vector clock advances. Listing 5.6 shows the `broadcast-from` function, which increments the sender's clock when creating a new message:

```
1  define (broadcast-from cfg actor value) :=
2    let {states := (get-cfg-states cfg);
3         clocks := (get-cfg-clocks cfg);
4         net := (get-cfg-net cfg);
5         actor-clock := (clocks-get clocks actor);
6         new-clock := (clock-tick actor-clock actor);
7         new-clocks := (clocks-set clocks actor new-clock);
8         msg := (msg actor value new-clock);
9         new-net := (MSet.insert msg net)}
10     (kfg states new-clocks new-net)
```

Snippet 5.6: Clock ticking in broadcast-from function

Message delivery combines payload and clock merge. When a message with timestamp clk arrives at a replica i, the framework merges the payload into local state and updates each vector clock.

5.4.2 Composition of State and Causality

The Backbone layer leverages the Theory layer by observing that vector clocks form a semilattice as well. The backbone combines the clock lattice with the payload's state lattice. We effectively work in the product lattice $S \times VC$: an element of the product is a pair (s, v) of a payload state and a vector clock. The join is defined as:

$$(s_1, v_1) \sqcup (s_2, v_2) = (s_1 \sqcup_S s_2, v_1 \sqcup_{VC} v_2),$$

that is, the payload states are joined by their $\sqcup$ and the clocks by pointwise maximum. This product construction is again a join-semilattice (it inherits ACI properties from each component). As a result, the entire backbone configuration space still satisfies the convergence axioms. Despite the added complexity of tracking clocks, the backbone layer's operations preserve the fundamental algebraic structure needed for SEC.

5.5 CRDT Adapters and Instantiation

The final layer of the architecture is where concrete CRDT implementations come into play. Instead of building each CRDT from scratch, the framework uses an adapter pattern to instantiate the generic payload/backbone tools with the details of a particular CRDT. A CRDT adapter is a module that connects a CRDT's native state and operations to the interfaces expected by the Payload (and optionally Backbone) framework.

Creating a new CRDT involves defining state and update types, implementing merge and apply functions, and proving the five semilattice axioms (A1–A5 from Chapter 4). These proofs are typically brief, reducing to algebraic properties of the chosen data types (e.g., GCounter proofs reduce to properties of max). Once proven, the framework automatically guarantees SEC via the theory/adapter pattern, eliminating the need to re-prove convergence for each CRDT.

Every CRDT has a state type, merge operation, update function, and initial state. This connection enables proof reuse and eases the addition of new CRDTs. Product constructions inherit axiom proofs automatically. Because the framework is implemented in Athena, specifications are executable: the same adapter used in proofs can generate runnable instances for testing and demonstration. See Chapter 6 for concrete examples.

6 Verifying CRDT Implementations

6.1 Introduction

Once established the foundation (Chapter 4) and the unified framework architecture (Chapter 5), we present different CRDTs implementations that demonstrate the verification capability of our system.

Note: This chapter presents the main theorems for each CRDT. The complete verified implementations, including all lemmas and helper proofs, are available in the repository.[1]

For each CRDT, we demonstrate the following classes of verified properties:

1. **Monotonicity properties:** Operations and merges never decrease over state components. This ensures both local updates and merge operations move upward in the lattice partial order.

2. **Progress properties:** Updates applied at one replica remain visible after merging with concurrent states. Formally, $merge(s, apply(s, v)) = apply(s, v)$.

3. **Behavioral guarantees:** CRDT-specific properties like add-wins semantics (OR-Set), deterministic tie-breaking (LWW-Register), or component independence (PN-Counter).

We organize our presentation into three representative CRDTs demonstrating different verification challenges:

- **PN-Counter** (Section 6.2) – a state-based CRDT with proven properties demonstrating monotonicity and progress guarantees.

[1] `https://github.com/FLAGlab/Athena-CRDTs/tree/verified-implementation`

- **LWW Register** (Section 6.3) – a causal CRDT showing how vector clocks enable deterministic concurrent write resolution.
- **OR-Set** (Section 6.4) – a grow-only set with add-wins semantics, demonstrating complex set membership reasoning and re-addition correctness.

Note: Complete implementations for all six CRDTs (G-Counter, G-Set, PN-Counter, 2P-Set, OR-Set, and LWW-Register) are available in the code repository. We present the most representative here.

6.2 PN-Counter: A Showcase of Verified Properties

We focus on the PN-Counter (Positive-Negative Counter) as a representative state-based CRDT. PN-Counter supports both increments and decrements through a pair of GCounters: one for positive updates (P) and one for negative updates (N). The observed value is computed as $P - N$.

This design illustrates the state (lattice position) grows monotonically under the merge operation, but the computed value (obtained through subtraction) may decrease. Formal verification is essential to ensure that this separation between state semantics and value computation does not introduce unintended behavior.

6.2.1 Specification and Properties

State representation.

$$\text{State} = (\text{Ide} \to \mathbb{N}) \times (\text{Ide} \to \mathbb{N}) = P \times N$$

A pair of finite maps, each associating replicas with natural number counts.

Operations. The PN-Counter supports the following operations:

- *increment*: $\mathit{increment}((P, N), r) = (P[r \mapsto P(r) + 1], N)$
- *decrement*: $\mathit{decrement}((P, N), r) = (P, N[r \mapsto N(r) + 1])$
- *merge*: $\mathit{merge}((P_1, N_1), (P_2, N_2)) = (\max_r(P_1(r), P_2(r)), \max_r(N_1(r), N_2(r)))$

- *value*: $value((P, N)) = \sum_r P(r) - \sum_r N(r)$

The merge operation applies componentwise maximum to both the positive and negative components. The value function performs arithmetic subtraction, which may decrease even though the underlying state (the pair of maps) grows monotonically.

PN-Counter's theorem collection. The PN-Counter specification contains **22 proven theorems**, among which we highlight monotonicity, progress preservation, and semantic distinction:

Category	**Representative Theorems**
Structural	`pnc-merge-is-pointwise-max`, `merge-commutative`
Component Monotonicity	`positive-component-monotonic`, `negative-component-monotonic`
Update Monotonicity	`increment-pos-monotonic`, `increment-neg-monotonic`
Progress	`increment-increases-P`, `decrement-increases-N`

Table 6.1: Representative theorems from PN-Counter's verified properties. Progress properties (`increment-increases-P`, `decrement-increases-N`) demonstrate that updates increase their respective components, while merge preservation properties (`merge-preserves-pos-update`, `merge-preserves-neg-update`) ensure updates survive merging.

These theorems demonstrate different aspects of CRDT correctness: (1) *component monotonicity* ensures merge operations never decrease state components and (2) *progress preservation* ensures local updates are preserved after merging with concurrent states.

First, Listing 6.1 establishes component monotonicity:

```
1 conclude positive-component-monotonic :=
2   (forall s1 s2 . (N.<= (first s1) (first (pnc-merge s1 s2))))
3   pick-any s1:PNCState s2:PNCState
```

```
    (!two-cases
      assume case1 := ((first s2) < (first s1))
        conclude ((first s1) N.<= (first (pnc-merge s1 s2)))
          let {eq1 := (!chain [(first (pnc-merge s1 s2))
                       = (N.Max (first s1) (first s2)) [pnc-merge-is-
                           pointwise-max]
                       = (first s1) [N.Max.less2]]);
               refl := (!uspec N.Less=.reflexive (first s1))}
            (!rcong refl ((first s1) N.<= (first (pnc-merge s1 s2))))
      assume case2 := (~ ((first s2) < (first s1)))
        conclude ((first s1) N.<= (first (pnc-merge s1 s2)))
          (!chain-> [case2
                     ==> ((first s1) N.<= (first s2)) [N.Less=.trichotomy2]
                     ==> ((first s1) N.<= (N.Max (first s1) (first s2)))
                     ==> ((first s1) N.<= (first (pnc-merge s1 s2)))]))
```

Snippet 6.1: PN-Counter: Component monotonicity

This proves the positive component never decreases through merge.

Second, Listing 6.2 demonstrates progress preservation:

```
conclude merge-preserves-pos-update :=
  (forall s s2 d .
    (first (pnc-apply-update s (inc-pos d))) N.<=
    (first (pnc-merge (pnc-apply-update s (inc-pos d)) s2)))
  pick-any s:PNCState s2:PNCState d:N
    let {s-updated := (pnc-apply-update s (inc-pos d))}
      (!uspec (!uspec positive-component-monotonic s-updated) s2)
```

Snippet 6.2: PN-Counter: Progress preservation (updates survive merging)

This establishes that local updates survive merge operations. The proof instantiates the component monotonicity theorem twice: first specializing it to the updated state `s-updated`, then to the arbitrary state `s2`. If the positive component never decreases, then an increment to that component will remain visible after any subsequent merge.

Third, Listing 6.3 formalizes semantic distinction:

```
conclude state-grows-value-may-decrease :=
  (forall s1 s2 .
    ((N.<= (first s1) (first s2)) ==>
     (N.<= (second s1) (second s2)) ==>
     (pnc-merge s1 s2) = s2))
  pick-any s1:PNCState s2:PNCState
    assume h1 := ((first s1) N.<= (first s2))
      assume h2 := ((second s1) N.<= (second s2))
        let {max-f := (!two-cases ...);
             max-s := (!two-cases ...)}
          (!chain [(pnc-merge s1 s2)
                   = (pr (N.Max (first s1) (first s2))
                         (N.Max (second s1) (second s2)))
                   = (pr (first s2) (second s2)) [max-f max-s]
                   = s2])
```

Snippet 6.3: PN-Counter: State grows but value may decrease

Together, these three theorems (Listings 6.1, 6.2, and 6.3) establish core verification guarantees: monotonicity ensures information is never lost, progress preservation ensures updates propagate reliably, and semantic distinction formalizes that lattice order and computed value have fundamentally different properties.

The progress preservation property (Listing 6.2) proves that after a replica increments its positive component, merging with any other state will preserve that increment. This implements the formal requirement $merge(s, apply(s, v)) = apply(s, v)$ and ensures that any replica does not lose its local updates while synchronizing.

6.3 LWW Register: Deterministic Concurrent Write Resolution via Vector Clocks

LWW Register (Last-Write-Wins Register) addresses a fundamental problem in replicated state: when two replicas perform concurrent writes to the same register, how should the system deterministically select one value? The LWW Register uses vector clocks to establish a happens-before relation, enabling the system to order writes and apply a last-write-wins policy deterministically.

6.3.1 The Challenge

Consider replicas R_1 and R_2 concurrently writing different values to a register. The system must resolve these concurrent writes to a single value. Three approaches are possible:

- **Arbitrary selection**: Choose one value randomly (no determinism).
- **Merge-based**: Combine both writes (not applicable for scalar values).
- **Last-write-wins**: Order writes by causality; apply whichever write came last (LWW Register).

LWW Register implements last-write-wins through vector clocks, which provide a total order on causally comparable events. When two writes are concurrent (not ordered by causality), the register applies a deterministic tie-breaking rule (lexicographic order on values). This requires the causal metadata provided by the Backbone framework (Chapter 5).

6.3.2 State and Operations

State representation. The LWW Register state consists of a value paired with its vector clock:

$$\text{State} = (\text{value}, \text{clock}) \quad \text{where} \quad \text{value} \in \mathbb{V} \text{ and clock} \in \text{Clock}$$

where $\mathbb{V}$ is the domain of values (typically identifiers or scalars).

The clock records the logical timestamp of the most recent write. An element is visible through the register lookup operation.

Key operations. The LWW Register supports the following operations:

- *read*: $read((v, c)) = v$
- *write*: $write((v_1, c_1), v_2, c_2) = (v_2, c_2)$
- *merge*: $merge((v_1, c_1), (v_2, c_2)) = \text{if } c_1 \succ c_2 \text{ then } (v_1, c_1) \text{ else } (v_2, c_2)$
- *tick*: $tick(c, r) = c[r \mapsto c(r) + 1]$

Merge compares vector clocks: if one strictly dominates the other, that value is retained. If clocks are incomparable (concurrent writes), tie-breaking selects the lexicographically smaller value. The tick operation increments the local replica's clock before each write, maintaining causality.

Verified properties. LWW Register's specification (188 lines in Athena) contains proofs of the following categories:

Category	**Representative Theorems**
Merge Structure	Commutativity, associativity, idempotence of merge
Causality	Happens-before order ensures write precedence
Convergence	Sequential writes guarantee convergence; concurrent writes use tie-breaking

Table 6.2: Verified property categories for LWW Register

Clock and value preservation properties. The LWW Register specification includes proven theorems for update and merge operations. We present two representative examples demonstrating how the register preserves vector clock structure and deterministic merge semantics.

First, Listing 6.4 shows the structure of the update operation:

```
declare lww-update: [LWWState Update ActorId] -> LWWState

assert* lww-update-def :=
  [((lww-update s (write v) a) =
    (pr v (clock-tick (second s) a)))]
```

Snippet 6.4: LWW Register: Update definition

This definition shows that updates pair the new value with a ticked clock.

Second, Listing 6.5 proves that merge joins the vector clocks:

```
conclude merge-clock-is-join :=
  (forall s1 s2 .
    (second (lww-merge s1 s2)) = (clock-join (second s1) (second s2)))
pick-any s1:LWWState s2:LWWState
  (!chain [(second (lww-merge s1 s2))
         = (second (pr (lww-select-value (first s1) (second s1)
                                         (first s2) (second s2))
                       (clock-join (second s1) (second s2))))
           [lww-merge-def]
         = (clock-join (second s1) (second s2)) [Pr.pr-second]])
```

Snippet 6.5: LWW Register: Merge joins clocks

This establishes that merge computes the join (semilattice supremum) of the input clocks, which is the foundation for causal consistency.

These two theorems (Listings 6.4 and 6.5) together ensure that LWW Register operations are well-formed: updates introduce new values with properly ticked clocks, and merges preserve the vector clock lattice structure that enables causal ordering.

6.4 OR-Set: Composing CRDT State with Vector Clocks

The Observed-Remove Set (OR-Set) demonstrates a more complex interaction between CRDT semantics and causal ordering. Unlike state-based CRDTs (PN-Counter, G-Set), OR-Set requires

vector clocks to implement add-wins semantics: when an add and remove operation occur concurrently, the add wins. This requires tracking causality to distinguish between "remove followed by concurrent add" (element present) versus "add followed by causally-later remove" (element absent).

6.4.1 The Challenge: Re-addition Without Causality

Consider two replicas performing operations on element x:

1. R_1 adds x at time t_1
2. R_2 removes x at time t_2 (concurrent with t_1)
3. R_1 adds x again at time t_3 (after seeing the removal)

Without causality tracking, a pure state-based merge cannot distinguish the re-addition at t_3 from the original add at t_1. The OR-Set solves this by pairing each element with a unique tag generated from the replica's vector clock, creating a *causal CRDT* that merges both set state and clock state.

6.4.2 State Representation and Extended Merge

OR-Set state pairs a set of tagged elements with a vector clock:

$$\text{ExtendedORSetState} = (\text{ORSetState}, \text{Clock})$$

where ORSetState contains sets of adds (tagged elements) and removes (tags to tombstone).

The extended merge operation must merge *both* the OR-Set state and the vector clocks, while maintaining the semilattice properties. This requires proving that the composition of two semilattices (set union semilattice and clock join semilattice) forms a valid semilattice.

Proof: Extended merge commutativity. Listing 6.6 demonstrates how CRDT merge and vector clock join compose:

```
conclude extended-merge-commutative :=
  (forall s1 s2 .
    (orset-merge-with-context s1 s2) = (orset-merge-with-context s2 s1))
pick-any s1:ExtendedORSetState s2:ExtendedORSetState
    let {state1 := (first s1);
         clock1 := (second s1);
         state2 := (first s2);
         clock2 := (second s2);
         merged-state-12 := (orset-merge state1 state2);
         merged-state-21 := (orset-merge state2 state1);
         joined-clock-12 := (clock-join clock1 clock2);
         joined-clock-21 := (clock-join clock2 clock1);
         state-comm := (!chain [(orset-merge state1 state2)
                                = (orset-merge state2 state1)
                                    [Theory.merge-commutative-holds]]);
         clock-comm := (!chain [(clock-join clock1 clock2)
                                = (clock-join clock2 clock1)
                                    [VectorClocks.Properties.clock-join-
                                        commutative]]);
         result12 := (pr merged-state-12 joined-clock-12);
         result21 := (pr merged-state-21 joined-clock-21);
         pair-eq := (!chain [(pr merged-state-12 joined-clock-12)
                             = (pr merged-state-21 joined-clock-21)
                                 [state-comm clock-comm]])}
      (!chain [(orset-merge-with-context s1 s2)
               = result12
               = result21  [pair-eq]
               = (orset-merge-with-context s2 s1)])
```

Snippet 6.6: OR-Set: Composing CRDT and vector clock commutativity

This proof combines the proven commutativity of OR-Set merge (from the CRDT specification)

with the proven commutativity of vector clock join (from the framework's vector clock theory). The key insight is in the `pair-eq` step, which uses both `state-comm` and `clock-comm` to establish equality of the paired result.

6.5 The Adapter Pattern in Practice

Each CRDT is implemented by the adapter pattern shown in Figure 6.1. This section details how adapters modularize verification.

6.5.1 Adapter Structure

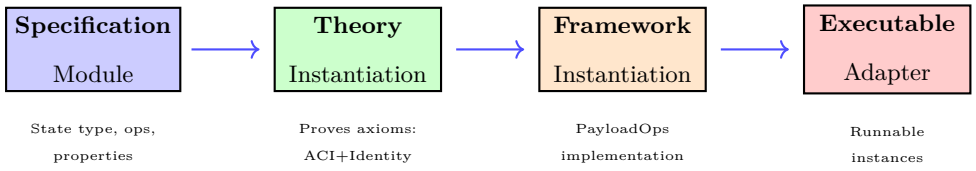


Figure 6.1: CRDT Adapter Template: The four-stage progression from formal specification to executable adapter. Each stage builds on previous verification: *Specification module* defines state and operations; *Theory instantiation* proves satisfaction of semilattice axioms (commutativity, associativity, idempotence, identity); *Framework instantiation* provides `PayloadOps` implementations; *Executable adapter* uses configuration machinery for testing and demonstration.

Example: PN-Counter Adapter. The PN-Counter adapter (197 lines) consists of:

1. **Specification module**: Defines state type, operations, and proves some theorems
2. **Theory instantiation**: Proves ACI axioms for componentwise merge by inheriting GCounter proofs
3. **Framework instantiation**: Registers with PayloadFramework, inheriting convergence guarantees
4. **Demo module**: Instantiates replicas, performs operations, demonstrates convergence

6.5.2 Inheriting Properties via Adapters

The theory/adapter pattern works by creating a renaming that maps CRDT operations to abstract semilattice operations, then automatically inheriting theorems. Listing 6.7 shows a concrete example from the GSet (Grow-only Set) implementation:

```
module GSetSpec {
  define-sort GSState := (Set.Set Ide)
  declare gs-merge: [GSState GSState] -> GSState
  declare gs-initial: GSState

  assert* gs-merge-is-union:=[((gs-merge ?s1 ?s2)=(Set.union ?s1 ?s2))]

  module Theory {
    conclude merge-commutative :=
      (forall s1 s2 . (gs-merge s1 s2) = (gs-merge s2 s1))
      pick-any s1:GSState s2:GSState
        (!chain [(gs-merge s1 s2)
                = (Set.union s1 s2) [gs-merge-is-union]
                = (Set.union s2 s1) [Set.union-commutes]
                = (gs-merge s2 s1) [gs-merge-is-union]])

    # Create adapter to abstract SemilatticeTheory
    define GSet-To-Semilattice :=
      (renaming |{SemilatticeTheory.join := gs-merge,
                  SemilatticeTheory.bot := gs-initial}|)

    # Inherit derived theorems automatically
    define gs-absorption :=
      (get-property SemilatticeTheory.absorption
                    GSet-To-Semilattice
                    SemilatticeTheory.theory)}}
```

Snippet 6.7: GSet adapter: inheriting semilattice theorems via renaming

Notice that after proving only the core ACI axioms and establishing the renaming, all other properties (absorption, monotonicity, convergence) are automatically available. The same `get-property` pattern applies to inherited theorems without requiring any manual proving.

6.5.3 Executable Demonstration

The adapter pattern is not merely a theoretical abstraction; it produces runnable code. Listing 6.8 shows a demonstration script for PN-Counter that instantiates three replicas, performs concurrent increments and decrements, and verifies convergence:

```
load "../crdts/pncounter/pncounter_payload_adapter"
open PNCounterAdapter
open Payload

# Create initial configuration with 3 replicas
define actors := ('r1 :: 'r2 :: 'r3 :: nil)
define cfg0 := (pnc-init actors)

# Replica r1 increments by 3, broadcast and deliver
define cfg1 := (pnc-broadcast cfg0 ('r1 :: nil)
                 (PNCounterSpec.inc-pos (S (S (S zero)))))
define cfg2 := (pnc-reduce cfg1)

# Replica r2 increments by 2, broadcast and deliver
define cfg3 := (pnc-broadcast cfg2 ('r2 :: nil)
                 (PNCounterSpec.inc-pos (S (S zero))))
define cfg4 := (pnc-reduce cfg3)

# Replica r3 decrements by 1, broadcast and deliver
define cfg5 := (pnc-broadcast cfg4 ('r3 :: nil)
                 (PNCounterSpec.inc-neg (S zero)))
define cfg6 := (pnc-reduce cfg5)

```

```
# Replica r1 decrements by 1, broadcast and deliver
define cfg7 := (pnc-broadcast cfg6 ('r1 :: nil)
                 (PNCounterSpec.inc-neg (S zero)))
define cfg-final := (pnc-reduce cfg7)

# Evaluate final configuration
(eval (am cfg-final))
```

Snippet 6.8: PN-Counter executable demo: concurrent operations and convergence

This demo script loads the verified PN-Counter specification. Each call to `pnc-broadcast` and `pnc-reduce` executes the merge operations whose properties were proven in Listings 6.1, 6.2, and 6.3. The final `eval` displays the converged state across all replicas: each replica's state shows $(P, N) = (3, 1)$ with computed value $3 - 1 = 2$, demonstrating that monotonic state growth (all components increased) coexists with value semantics (the value may decrease). The framework thereby provides both machine-checked formal guarantees and executable instances for testing—demonstrating that verification need not practicality sacrifice.

6.6 What We Verify: Summary

The implementations chapter demonstrates that our verification system can prove:

- **Structural properties**: Semilattice axioms (commutativity, associativity, idempotence) for merge operations
- **Monotonicity properties**: State components never decrease through merge operations (e.g., `positive-component-monotonic`, `negative-component-monotonic`)
- **Progress properties**: Local updates are preserved through merges (e.g., `merge-preserves-pos-update`)
- **Behavioral properties**: CRDT-specific invariants such as "state grows but value may decrease" (`state-grows-value-may-decrease`) and clock preservation in causal CRDTs (`merge-clock-is-join`)

Properties such as progress preservation are proven explicitly for PN-Counter (`merge-preserves-pos-update`, `merge-preserves-neg-update`), OR-Set (`merge-preserves-tag-sets`), G-Set (`merge-preserves-additions`), and 2P-Set (`remove-preserves-add-set`). For LWW-Register and G-Counter, progress follows from their proven merge properties via the theory/adapter pattern. In all scenarios the framework guarantees that updates applied at one replica remain visible after synchronization with concurrent replicas.

Each property is established through formal proof. All theorems presented in this chapter have been mechanically verified by Athena's proof engine.

7 Validation

This chapter validates that our framework verifies what is claimed: that CRDTs satisfying the five semilattice axioms are guaranteed to converge. Rather than relying on assertions or incomplete testing, we provide evidence that proofs are mechanically type-checked by the Athena proof engine before framework instantiation.

7.1 Proof Verification

All six CRDTs in the framework must prove the five semilattice axioms (A1–A5 from Chapter 4) before instantiation. This is a requirement enforced at compile-time by Athena's type system. Any missing or incorrect proof causes the framework to reject the CRDT module.

7.1.1 Coverage and Status

Table 7.1 shows the proof status for all six CRDTs:

All 30 axiom proofs have been mechanically type-checked by Athena. Each proof must be valid according to Athena's type system.

7.2 Connection to Convergence

The convergence guarantee in Chapter 4 establishes that any two replicas receiving the same set of updates in any order will converge to the same state, provided the CRDT satisfies axioms A1–A5. This theorem is proven once at the abstract level and then automatically applies to all CRDTs that prove these axioms.

Requiring mechanically type-checked axiom proofs ensures that:

Table 7.1: Axiom proof coverage (all 30 proofs verified)

CRDT	A1	A2	A3	A4	A5	Status
GCounter	✓	✓	✓	✓	✓	Verified
GSet	✓	✓	✓	✓	✓	Verified
PN-Counter	✓	✓	✓	✓	✓	Verified
Two-Phase Set	✓	✓	✓	✓	✓	Verified
OR-Set	✓	✓	✓	✓	✓	Verified
LWW-Register	✓	✓	✓	✓	✓	Verified
Total	**6**	**6**	**6**	**6**	**6**	**30 proofs**

- **No false convergence claims**: A CRDT cannot be instantiated without proving commutativity, associativity, and idempotence.
- **No weakened guarantees**: Missing or incomplete monotonicity proofs cause the framework to reject the CRDT.
- **Type-safe framework**: Only CRDTs with all five axioms proven are accepted by the PayloadFramework.

7.3 How Verification Works

The verification process follows three steps:

1. **Axiom proofs type-checked**: Athena mechanically verifies each proof of A1–A5, checking that types match and no circularity exists.
2. **Framework instantiation**: The PayloadFramework accepts the CRDT module only if all axioms are proven, allowing it to be instantiated as a concrete CRDT adapter.
3. **Convergence inherited**: The convergence theorem from SemilatticeTheory automatically applies because the instantiated CRDT satisfies all required axioms.

7.4 Implications

This approach provides several guarantees:

- **Properties are proven, not assumed**: The framework requires mechanically verified proofs of semilattice axioms (commutativity, associativity, idempotence) before CRDTs can be used.
- **Type-checked correctness**: Athena's proof engine verifies that each axiom proof is valid according to its type system, preventing invalid reasoning.
- **Reproducibility**: Any developer can verify the proofs by loading the CRDT specification files into Athena and observing that they type-check without errors.
- **Modular theorem inheritance**: CRDTs like GSet can use the adapter pattern to automatically inherit derived theorems (absorption, monotonicity) from the abstract semilattice theory without manual re-proof.

8 Discussion

8.1 Framework Evaluation

This section discusses the framework along three angles: what the architecture makes easy, what remains hard, and how it compares with established proof-assistant workflows.

Strengths

- **Modularity**: Clear separation of concerns ($Theory \rightarrow Payload \rightarrow Backbone$) enables independent reasoning. - **Proof reuse**: The theory/adapter pattern facilitates ACI and SEC proofs across instances; adapters are typically short and mechanical. - **Clarity**: Lattice laws are explicit; update monotonicity is stated once and reused. - **Executability**: Demos witness behaviour using the same definitions as the proofs.

Limitations

- **Manual adapters**: Adapters still require human-guided reductions to base theories (e.g., Max, Union). - **CRDT coverage**: Focused on core set/counter/register families; maps, sequences, and graphs are future work. - **Automation**: Tactic support is limited; structural equational proofs are concise but not fully automated.

8.2 Future Directions

- **Delta-CRDTs**: Add delta-join and anti-entropy schemas; reuse semilattice laws with payload-level deltas. - **Causal delivery guards**: The Backbone framework includes causal broadcast with vector clocks, but delivery precondition checking (ensuring messages are only applied

when causal dependencies are satisfied) is currently simplified. Implementing full causal delivery would strengthen operation-based CRDT guarantees. - **Automated adapter synthesis**: Derive adapter skeletons from type signatures and target base theories (Max/Union/Product).

Closing remark. The modular lattice-based approach scaled across six CRDTs with functional adapters and runnable demos. The next step is to broaden coverage while increasing automation to preserve this framework's style.

9 Conclusions and Future Work

This thesis developed a modular framework that allows to verify CRDTs in distributed systems. The central challenge is ensuring strong eventual consistency (SEC) in the presence of concurrency and asynchronous message delivery. By combining algebraic models with causal metadata, the system guarantees convergence without central coordination.

We began from first principles, formalising the semilattice model that underpins CRDT convergence. This abstraction provides the minimal requirements—commutativity, associativity, idempotence, and monotonicity—under which concurrent state updates can be merged deterministically. Chapter 4 captured these axioms as reusable Athena theories, which are later instantiated by specific CRDT designs.

Six CRDTs were implemented using this framework (Chapter 6). Each instantiates the abstract semantics, providing concrete merge and update operations, and discharging the required axioms. Formal proofs were constructed in Athena, demonstrating correctness by construction: once a CRDT conforms to the theory interface, convergence theorems follow automatically.

The framework was validated through both theoretical proofs and executable demonstrations (Chapter 7). Each layer was shown to satisfy its intended properties: Payload implements the semilattice model; Backbone preserves causal consistency; and together they yield SEC. All artifacts are open-source and publicly available on GitHub.

Limitations

This work focuses exclusively on *eventual convergence*. While this covers a large class of state-based CRDTs, other correctness dimensions—such as safety invariants, tombstone management, and performance optimizations are not addressed. The framework also assumes a reliable com-

munication layer; handling message loss or Byzantine faults is out of scope.

Bibliography

The references are sorted alphabetically by first author.

[1] Péter Bereczky, Dániel Horpácsi, and Simon Thompson. A formalisation of core erlang, a concurrent actor language. *Acta Cybernetica*, 26(3):373–404, 2024. DOI 10.14232/actacyb.298977. URL `https://cyber.bibl.u-szeged.hu/index.php/actcybern/article/view/4333`.

[2] Lyn Dupré. *BUGS in Writing: A Guide to Debugging Your Prose*. Pub-addison, revised edition, 1998. ISBN 0-201-37921-X.

[3] The Athena Foundation. Athena: A proof language for constructive verification. `https://github.com/AthenaFoundation/athena`. Accessed: 2025-12-06.

[4] Florian Jacob, Saskia Bayreuther, and Hannes Hartenstein. Byzantine behavior in crdts: Hashdag and matrix event graph. *SICHERHEIT 2022*, 2022. DOI 10.18420/SICHERHEIT2022_07. URL `http://dl.gi.de/handle/20.500.12116/40149`.

[5] Martin Kleppmann. Making crdts byzantine fault tolerant. In *9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC'22)*, pages 1–8, 2022. DOI 10.1145/3517209.3524042. URL `https://doi.org/10.1145/3517209.3524042`.

[6] Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, and Joseph M. Hellerstein. Katara: Synthesizing crdts with verified lifting. *Proc. ACM Program. Lang.*, 6:173, 2022. DOI 10.1145/3563336. URL `https://doi.org/10.1145/3563336`.

[7] Hongjin Liang and Xinyu Feng. Abstraction for conflict-free replicated data types. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language*

Design and Implementation, pages 636–650, 2021. DOI 10.1145/3453483.3454067. URL https://dl.acm.org/doi/10.1145/3453483.3454067.

[8] David R. Musser and Carlos A. Varela. Structured reasoning about actor systems. In *Proceedings of the 2013 Workshop on Programming based on Actors, Agents, and Decentralized Control*, pages 37–48, 2013. DOI 10.1145/2541329.2541334. URL https://dl.acm.org/doi/10.1145/2541329.2541334.

[9] Kartik Nagar and Suresh Jagannathan. Automated parameterized verification of crdts. *arXiv preprint*, 1905.05684v1, 2019. URL https://arxiv.org/abs/1905.05684v1.

[10] Abel Nieto, Léon Gondelman, Alban Reynaud, Amin Timany, and Lars Birkedal. Modular verification of op-based CRDTs in separation logic. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA2):1788–1816, October 2022. DOI 10.1145/3563351.

[11] Abel Nieto, Arnaud Daby-Seesaram, Léon Gondelman, Amin Timany, and Lars Birkedal. Modular verification of state-based crdts in separation logic. *arXiv preprint*, 2023.

[12] Kevin De Porre, Carla Ferreira, and Elisa Gonzalez Boix. Verifx: Correct replicated data types for the masses. *arXiv preprint*, 2207.02502, 2024. DOI 10.48550/arXiv.2207.02502. URL http://arxiv.org/abs/2207.02502.

[13] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems*, pages 386–400, 2011.

[14] William Strunk and E.B. White. *The Elements of Style*. Longman, fourth edition, 2000. ISBN 0-205-30902-X.

[15] Christian Tschudin. A connectionless grow-only set crdt. In *Proceedings of the 3rd International Workshop on Distributed Infrastructure for the Common Good*, pages 25–30, 2022. DOI 10.1145/3565383.3566110. URL https://dl.acm.org/doi/10.1145/3565383.3566110.

[16] Carlos Varela and Gul Agha. Programming dynamically reconfigurable open systems with salsa. *ACM SIGPLAN Notices*, 36(12):20–34, 2001. DOI 10.1145/583960.583964. URL `https://dl.acm.org/doi/10.1145/583960.583964`.

[17] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. Type-checking crdt convergence. *Proc. ACM Program. Lang.*, 7:162, 2023. DOI 10.1145/3591276. URL `https://doi.org/10.1145/3591276`.

[18] Peter Zeller, Annette Bieniusa, and Arnd Poetzsch-Heffter. Formal specification and verification of crdts. *Formal Techniques for Networked and Distributed Systems*, pages 33–48, 2014. DOI 10.1007/978-3-662-43613-4_3. URL `https://inria.hal.science/hal-01398007`.